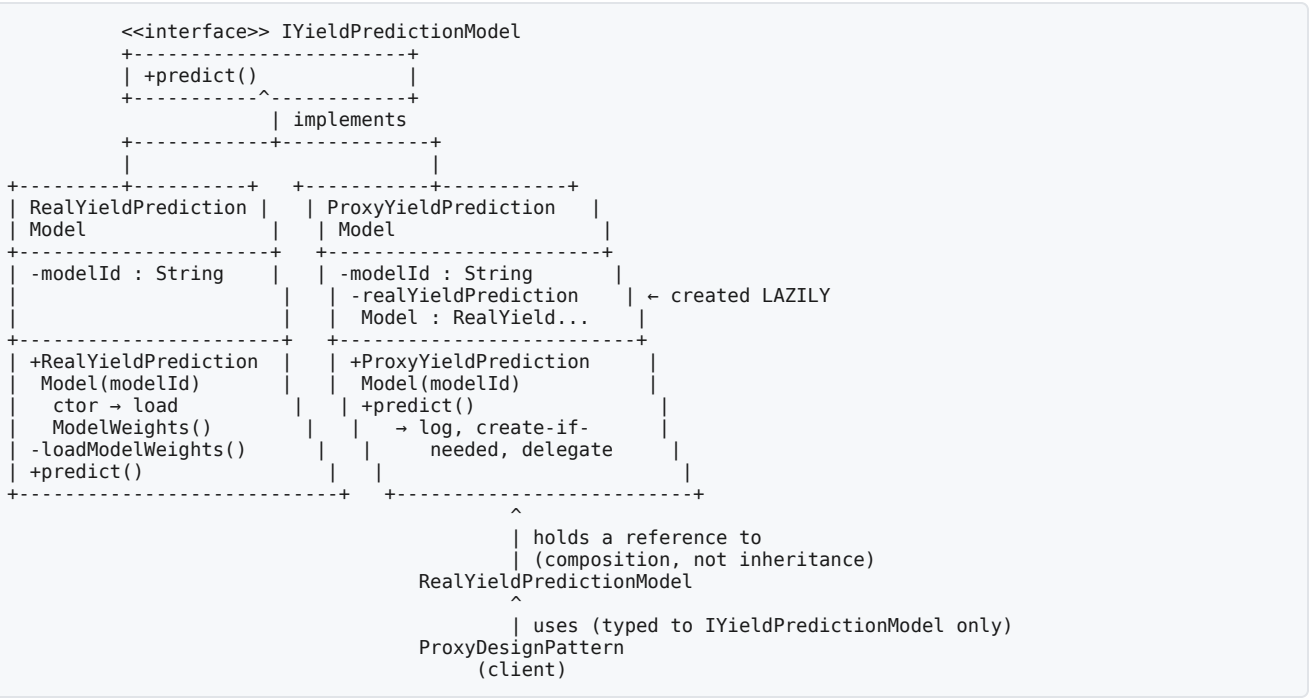
Proxy Design Pattern

Intent

Provide a **surrogate or placeholder for another object to control access to it**. The client talks to a stand-in (the *proxy*) that implements the same interface as the *real* object, so the client cannot tell the difference — and the proxy gets a chance to control access (lazy creation, logging, caching, guarding) before or after forwarding the call.

This module demonstrates the **virtual proxy** variant: the proxy defers creating the expensive real object until it is actually needed (first use), then reuses it for every later call.

UML class diagram



The players

```
subject/IYieldPredictionModel               the shared Subject interface — declares predict()
subject/realsubject/RealYieldPredictionModel the RealSubject — expensive object, loads its weights into memory
in its constructor
subject/proxy/ProxyYieldPredictionModel     the Proxy — implements IYieldPredictionModel, lazily owns/creates
a RealYieldPredictionModel
ProxyDesignPattern                          the client — programs only against IYieldPredictionModel, never
RealYieldPredictionModel
```

GoF role	Class in this module
Subject (common interface)	`IYieldPredictionModel` — declares `predict()`
RealSubject (does the real, expensive work)	`RealYieldPredictionModel` — loads its model weights into memory in its constructor, then predicts
Proxy (virtual proxy, controls access)	`ProxyYieldPredictionModel` — holds the model id; instantiates `RealYieldPredictionModel` on first `predict()`
Client	`ProxyDesignPattern.main()` — declares its variable as `IYieldPredictionModel`, constructs a `ProxyYieldPredictionModel`

Code walkthrough

`IYieldPredictionModel` — the Subject interface

```
package com.design.patterns.proxy.subject;

public interface IYieldPredictionModel {
    void predict();

}
```

- `package com.design.patterns.proxy.subject;` — lives in the neutral `subject` package, a sibling of both `realsubject` and `proxy`, because it belongs to neither implementation — it's the contract both share.
- `public interface IYieldPredictionModel` — declared as an interface, not an abstract class, because the only thing `RealYieldPredictionModel` and `ProxyYieldPredictionModel` need in common is the operation signature; there is no shared state or default behavior to inherit.
- `void predict();` — the single operation the client actually wants performed. It is the method the proxy must intercept in order to control access, and the method the real subject must eventually perform the real work.

`RealYieldPredictionModel` — the RealSubject

```
package com.design.patterns.proxy.subject.realsubject;

import com.design.patterns.proxy.subject.IYieldPredictionModel;

public class RealYieldPredictionModel implements IYieldPredictionModel {

    private final String modelId;

    public RealYieldPredictionModel(String modelId) {
        this.modelId = modelId;
        loadModelWeights();
    }

    private void loadModelWeights() {
        System.out.println("-----Loading model weights into memory: " + modelId);
    }

    @Override
    public void predict() {
        System.out.println("-----Running yield prediction with model: " + modelId);
    }
}
```

- `package ...realsubject;` — its own package, separate from `proxy`, so the two roles can never be confused by folder location; a reader can tell which class is "the real one" just from the path.
- `public class RealYieldPredictionModel implements IYieldPredictionModel` — implementing `IYieldPredictionModel` is what makes `RealYieldPredictionModel` substitutable for (and by) `ProxyYieldPredictionModel` — both satisfy the same contract the client depends on.
- `private final String modelId;` — the only state the real subject needs; `final` because a loaded model's identity (which trained checkpoint it represents) does not change after construction.
- `public RealYieldPredictionModel(String modelId) { this.modelId = modelId; loadModelWeights(); }` — the constructor is deliberately where the expensive work happens. This is what makes `RealYieldPredictionModel` "real" and "expensive": simply calling `new RealYieldPredictionModel(...)` pays the weight-loading cost immediately. This is the exact cost the proxy exists to defer.
- `private void loadModelWeights()` — kept as a separate private method (rather than inlined in the constructor) purely for readability — it names the expensive step so the constructor reads as "assign state, then load."
- `System.out.println("-----Loading model weights into memory: " + modelId);` — stands in for a real, expensive model-loading operation (deserializing gigabytes of trained parameters into memory); the leading dashes visually distinguish "real subject" output from "proxy" output when reading the console log.
- `@Override public void predict()` — the real, cheap-per-call operation once the object exists: it just runs inference using the already-loaded weights. No loading happens here — loading only ever happens once, in the constructor.

`ProxyYieldPredictionModel` — the Proxy

```
package com.design.patterns.proxy.subject.proxy;

import com.design.patterns.proxy.subject.IYieldPredictionModel;
import com.design.patterns.proxy.subject.realsubject.RealYieldPredictionModel;

public class ProxyYieldPredictionModel implements IYieldPredictionModel {

    private final String modelId;
    private RealYieldPredictionModel realYieldPredictionModel;

    public ProxyYieldPredictionModel(String modelId) {
        this.modelId = modelId;
    }
```

```
    @Override
    public void predict() {
        System.out.println("----Routing through model proxy");

        if (realYieldPredictionModel == null) {
            realYieldPredictionModel = new RealYieldPredictionModel(modelId);
        }
        realYieldPredictionModel.predict();
    }
}
```

- `package ...proxy;` — its own package too, mirroring `realsubject`, so the two roles are structurally symmetric and easy to tell apart.
- `public class ProxyYieldPredictionModel implements IYieldPredictionModel` — implementing the same `IYieldPredictionModel` interface is what lets the proxy stand in for the real subject: the client can hold an `IYieldPredictionModel` reference to either one, interchangeably.
- `private final String modelId;` — the proxy stores only the cheap identifying data (a model id), *not* the loaded weights. This is what makes construction of the proxy itself cheap.
- `private RealYieldPredictionModel realYieldPredictionModel;` — deliberately **not** `final` and starts out `null`. This field is the entire mechanism of the pattern: it is the cached handle to the real subject, absent until first needed.
- `public ProxyYieldPredictionModel(String modelId) { this.modelId = modelId; }` — the proxy's constructor does *not* construct a `RealYieldPredictionModel`. This is the crucial contrast with `RealYieldPredictionModel`'s own constructor: creating a proxy is cheap, and if `predict()` is never called, `RealYieldPredictionModel` — and its expensive weight load — never comes into existence at all.
- `@Override public void predict()` — this is where access control happens: every call funnels through the proxy first.
- `System.out.println("----Routing through model proxy");` — logs that the call passed through the access-control point, independent of whether this is the first call or a later one. This is a visible example of the "before/after forwarding" hook the proxy gets, beyond just lazy creation.
- `if (realYieldPredictionModel == null) { realYieldPredictionModel = new RealYieldPredictionModel(modelId); }` — the lazy-initialization gate: creates (and pays the weight-loading cost for) the real subject only the first time it's genuinely needed, then caches it in the field so the cost is never paid twice.
- `realYieldPredictionModel.predict();` — delegates the actual work to the real subject by composition (the proxy *holds* a `RealYieldPredictionModel`, it does not `extends RealYieldPredictionModel`), which is what lets the proxy control *when* delegation starts without owning any of the real subject's implementation.

`ProxyDesignPattern` — the client

```
package com.design.patterns.proxy;

import com.design.patterns.proxy.subject.IYieldPredictionModel;
import com.design.patterns.proxy.subject.proxy.ProxyYieldPredictionModel;

public class ProxyDesignPattern {

    public static void main(String[] args) {
        System.out.println("Proxy Design Pattern");

        IYieldPredictionModel model = new ProxyYieldPredictionModel("corn-yield-forecaster-v3");

        model.predict();

        model.predict();
    }
}
```

- `import ...subject.IYieldPredictionModel;` and `import ...subject.proxy.ProxyYieldPredictionModel;` — note there is **no import of `RealYieldPredictionModel`**. The client's source code has no way to reference the real subject even if it wanted to — it is not on the client's compile-time surface at all.
- `System.out.println("Proxy Design Pattern");` — a banner line identifying which demo is running, printed before any pattern activity.
- `IYieldPredictionModel model = new ProxyYieldPredictionModel("corn-yield-forecaster-v3");` — the client constructs a `ProxyYieldPredictionModel`, but declares the variable's *static type* as `IYieldPredictionModel`. From this line onward, every use of `model` is polymorphic — the client only ever knows it's talking to "something that can `predict()`." This is the substitutability the Subject interface exists to guarantee, and it's what makes the proxy transparent to the client.
- `model.predict();` (first call) — triggers the proxy's lazy-creation gate: `realYieldPredictionModel` is `null`, so `ProxyYieldPredictionModel` builds a `RealYieldPredictionModel` here (paying the weight-loading cost) and then delegates.
- `model.predict();` (second call) — the proxy's `realYieldPredictionModel` field is now non-null, so this call skips creation entirely and delegates straight to the already-loaded `RealYieldPredictionModel`. Calling `predict()` twice with only one `Loading model weights into memory` line in the output is the observable proof that the proxy is doing its job.

Why these design decisions

Why an interface (`IYieldPredictionModel`) instead of a base class?

`RealYieldPredictionModel` and `ProxyYieldPredictionModel` share no state and no default logic — only the operation signature. An interface is the minimal contract that gives the client substitutability without forcing any inheritance relationship between the two implementations.

Why does `RealYieldPredictionModel` do its expensive work in the constructor rather than in `predict()`?

To make the cost model explicit and testable: "an instance of `RealYieldPredictionModel` exists" and "the expensive work has been done" become the same fact. That, in turn, is exactly what lets `ProxyYieldPredictionModel` control the cost — by controlling *when a `RealYieldPredictionModel` is instantiated at all*, it controls when the expensive work happens, with no extra bookkeeping (no separate "isLoaded" flag needed anywhere).

Why does `ProxyYieldPredictionModel` hold a `RealYieldPredictionModel` field instead of extending `RealYieldPredictionModel`?

Composition, not inheritance, is what allows the proxy to control *whether and when* the real object exists in the first place. If `ProxyYieldPredictionModel extended RealYieldPredictionModel`, a `RealYieldPredictionModel` (and its expensive constructor work) would necessarily be built the moment a `ProxyYieldPredictionModel` is built — defeating the entire purpose of a virtual proxy.

Why is `realYieldPredictionModel` left non-`final` and nullable, while `modelId` in both classes is `final`?

`modelId` is fixed identity, known at construction, so it is `final` in both classes. `realYieldPredictionModel` in the proxy is the one piece of state that is deliberately absent at construction and populated later exactly once — that's the lazy-init idiom, and `final` would make it impossible.

Why does the client declare `IYieldPredictionModel model = new ProxyYieldPredictionModel(...)` instead of `ProxyYieldPredictionModel model = ...`?

This is the detail that actually proves the pattern is in effect rather than merely present in the file tree. Typing the variable to the interface means the client's code is identical to what it would be if it were talking directly to a `RealYieldPredictionModel` — the proxy is a drop-in, undetectable substitute. Typing it to `ProxyYieldPredictionModel` would still compile, but would weaken the demonstration by exposing the concrete proxy type to the client.

Why does `predict()` print a log line on every call, not just the first?

To make the access-control point observable on every invocation, not only on the interesting (first) one — matching how real proxies (Hibernate lazy entities, Spring AOP, `java.lang.reflect.Proxy`) typically intercept *every* call, even when most of their work (like the load-once cache check) is only expensive the first time.

Execution flow trace

```
ProxyDesignPattern.main
    │
    ├── println("Proxy Design Pattern")
    │
    ├── IYieldPredictionModel model = new ProxyYieldPredictionModel("corn-yield-forecaster-v3")
    │         └── ProxyYieldPredictionModel ctor stores modelId only — no RealYieldPredictionModel yet,
realYieldPredictionModel == null
    │
    ├── model.predict()                              [1st call]
    │         └── ProxyYieldPredictionModel.predict()
    │                   ├── println("----Routing through model proxy")
    │                   ├── realYieldPredictionModel == null → true
    │                   │       └── new RealYieldPredictionModel("corn-yield-forecaster-v3")
    │                   │               └── ctor → loadModelWeights()
    │                   │                       └── println("-----Loading model weights into memory: corn-
yield-forecaster-v3")
    │                   └── realYieldPredictionModel.predict()
    │                           └── println("-----Running yield prediction with model: corn-yield-forecaster-v3")
    │
    └── model.predict()                              [2nd call]
              └── ProxyYieldPredictionModel.predict()
                        ├── println("----Routing through model proxy")
                        ├── realYieldPredictionModel == null → false (cached from 1st call)
                        │       └── (creation skipped — no reload)
                        └── realYieldPredictionModel.predict()
                                └── println("-----Running yield prediction with model: corn-yield-forecaster-v3")
```

The trace shows the pattern's payoff directly: `loadModelWeights()` runs exactly once across two `predict()` calls, because the proxy — not the client — decides when the real subject is built.

Expected output

Captured from an actual run of `java -cp target/classes com.design.patterns.proxy.ProxyDesignPattern`:

```
Proxy Design Pattern
----Routing through model proxy
-----Loading model weights into memory: corn-yield-forecaster-v3
-----Running yield prediction with model: corn-yield-forecaster-v3
----Routing through model proxy
-----Running yield prediction with model: corn-yield-forecaster-v3
```

Note the proof in the output: two `predict()` calls, but `Loading model weights into memory` appears only once — the proxy deferred creation to first use and reused the loaded subject afterwards.

How to run

From inside this module directory (`Structural/Proxy_Design_Pattern`):

```
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile
java -cp target/classes com.design.patterns.proxy.ProxyDesignPattern
```